

```

import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Preprocessing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder

# Models
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

# Evaluation
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    classification_report,
    RocCurveDisplay
)

# Imbalance Handling
from imblearn.over_sampling import SMOTE

```

```

df = pd.read_csv('loan_prediction.csv')
df.head()

```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIn
0	LP001002	Male	No	0	Graduate	No	5849	
1	LP001003	Male	Yes	1	Graduate	No	4583	15
2	LP001005	Male	Yes	0	Graduate	Yes	3000	
3	LP001006	Male	Yes	0	Not Graduate	No	2583	25
4	LP001008	Male	No	0	Graduate	No	6000	

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```

df.info()
df.describe()

```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 614 entries, 0 to 613
```

```
Data columns (total 13 columns):
```

#	Column	Non-Null Count	Dtype
0	Loan_ID	614 non-null	object
1	Gender	601 non-null	object
2	Married	611 non-null	object
3	Dependents	599 non-null	object
4	Education	614 non-null	object
5	Self_Employed	582 non-null	object
6	ApplicantIncome	614 non-null	int64
7	CoapplicantIncome	614 non-null	float64
8	LoanAmount	592 non-null	float64
9	Loan_Amount_Term	600 non-null	float64
10	Credit_History	564 non-null	float64
11	Property_Area	614 non-null	object
12	Loan_Status	614 non-null	object

```
dtypes: float64(4), int64(1), object(8)
```

```
memory usage: 67.2+ KB
```

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History
count	614.000000	614.000000	592.000000	600.00000	564.000000
mean	5403.459283	1621.245798	146.412162	342.00000	0.842199
std	6109.041673	2926.248369	85.587325	65.12041	0.364878
min	150.000000	0.000000	9.000000	12.00000	0.000000
25%	2877.500000	0.000000	100.000000	360.00000	1.000000
50%	3812.500000	1188.500000	128.000000	360.00000	1.000000
75%	5795.000000	2297.250000	168.000000	360.00000	1.000000
max	81000.000000	41667.000000	700.000000	480.00000	1.000000



```
df.isnull().sum()
```

	0
Loan_ID	0
Gender	13
Married	3
Dependents	15
Education	0
Self_Employed	32
ApplicantIncome	0
CoapplicantIncome	0
LoanAmount	22
Loan_Amount_Term	14
Credit_History	50
Property_Area	0
Loan_Status	0

```
dtype: int64
```

```
df.columns = df.columns.str.strip()
```

```
print(df.columns.tolist())
```

```
['Loan_ID', 'Gender', 'Married', 'Dependents', 'Education', 'Self_Employed', 'ApplicantIncome', 'C
```

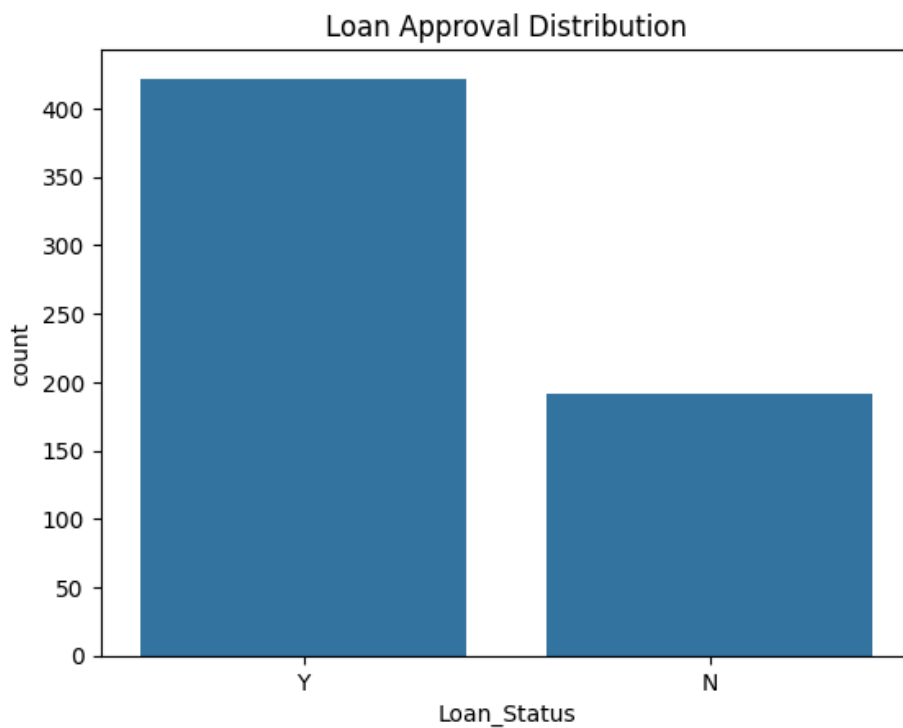
```
' loan status'
```

```
' loan status'
```

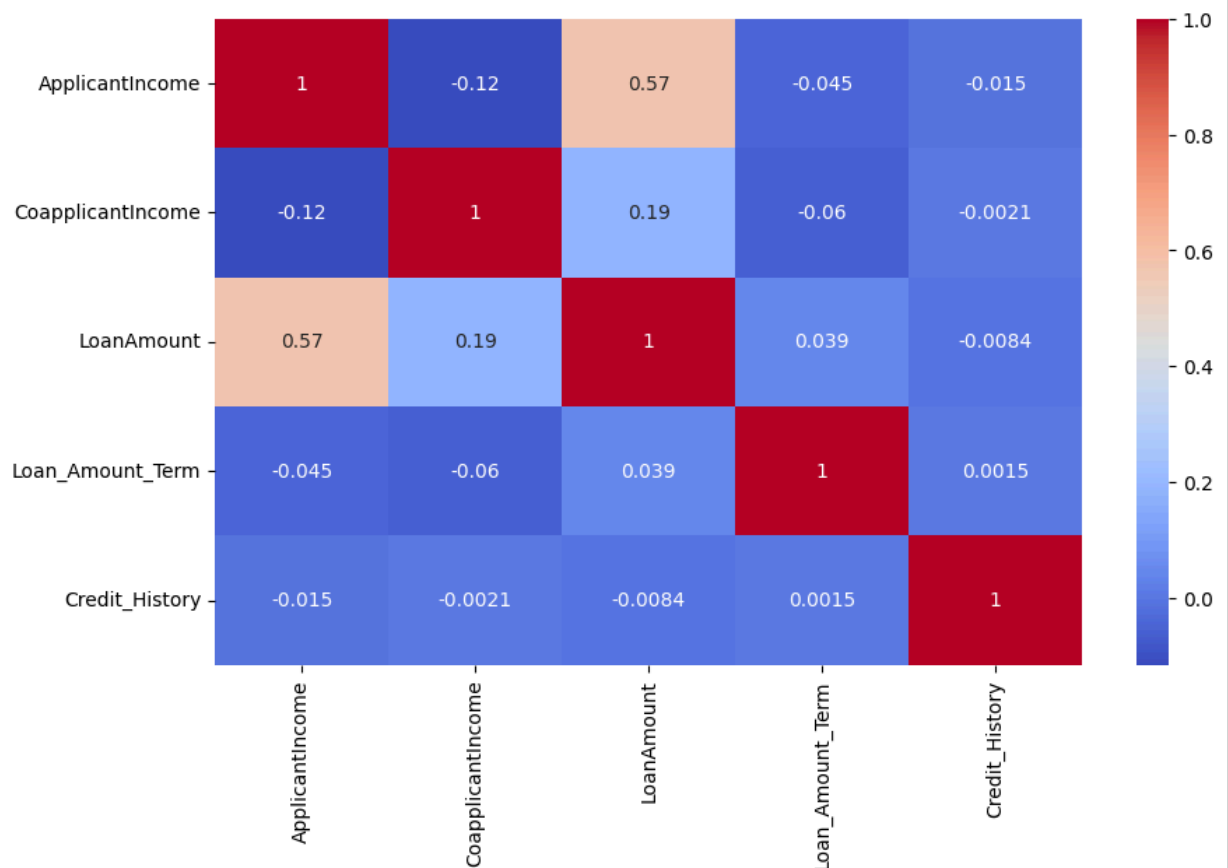
```
df.columns = df.columns.str.strip()  
print(df.columns)
```

```
Index(['Loan_ID', 'Gender', 'Married', 'Dependents', 'Education',  
      'Self_Employed', 'ApplicantIncome', 'CoapplicantIncome', 'LoanAmount',  
      'Loan_Amount_Term', 'Credit_History', 'Property_Area', 'Loan_Status'],  
      dtype='object')
```

```
sns.countplot(x='Loan_Status', data=df)  
plt.title('Loan Approval Distribution')  
plt.show()
```



```
plt.figure(figsize=(10,6))  
sns.heatmap(df.corr(numeric_only=True),  
            annot=True,  
            cmap='coolwarm')  
plt.show()
```



```
X = df.drop('Loan_Status', axis=1)
y = df['Loan_Status']
```

```
y = y.map({'Approved':1, 'Rejected':0})
```

```
numeric_features = X.select_dtypes(
    include=['int64', 'float64']
).columns

categorical_features = X.select_dtypes(
    include=['object']
).columns
```

```
numeric_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])
```

```
categorical_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore'))
])
```

```
preprocessor = ColumnTransformer([
    ('num', numeric_transformer, numeric_features),
    ('cat', categorical_transformer, categorical_features)
])
```

```
print(df['Loan_Status'].unique())
```

```
['Y' 'N']
```

```
df['Loan_Status'] = df['Loan_Status'].str.strip()

print(df['Loan_Status'].unique())
```

```
['Y' 'N']
```

```
y = df['Loan_Status'].map({
    'Y': 1,
    'N': 0
})
```

```
print(y.isna().sum())
```

```
0
```

```
print(y.value_counts())
```

```
Loan_Status
1    422
0    192
Name: count, dtype: int64
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
X_train_processed = preprocessor.fit_transform(X_train)
```

```
smote = SMOTE(random_state=42)
```

```
X_train_smote, y_train_smote = smote.fit_resample(
    X_train_processed,
    y_train
)
```

```
print("Before:", y_train.value_counts())
print("After:", pd.Series(y_train_smote).value_counts())
```

```
Before: Loan_Status
1    337
0    154
Name: count, dtype: int64
After: Loan_Status
1    337
0    337
Name: count, dtype: int64
```

```
lr = LogisticRegression(max_iter=1000)
```

```
lr.fit(X_train_smote, y_train_smote)
```

```
X_test_processed = preprocessor.transform(X_test)
```

```
y_pred_lr = lr.predict(X_test_processed)
```

```
y_prob_lr = lr.predict_proba(X_test_processed)[: ,1]
```

```
print("Precision:",
      precision_score(y_test,y_pred_lr))

print("Recall:",
      recall_score(y_test,y_pred_lr))

print("F1 Score:",
      f1_score(y_test,y_pred_lr))

print("ROC-AUC:",
      roc_auc_score(y_test,y_prob_lr))
```

```
Precision: 0.8631578947368421
Recall: 0.9647058823529412
F1 Score: 0.9111111111111111
ROC-AUC: 0.8687306501547988
```

```
rf = RandomForestClassifier(
    n_estimators=200,
    random_state=42
)

rf.fit(X_train_smote, y_train_smote)

y_pred_rf = rf.predict(X_test_processed)

y_prob_rf = rf.predict_proba(
    X_test_processed
)[: ,1]
```

```
print("Precision:",
      precision_score(y_test,y_pred_rf))

print("Recall:",
      recall_score(y_test,y_pred_rf))

print("F1 Score:",
      f1_score(y_test,y_pred_rf))

print("ROC-AUC:",
      roc_auc_score(y_test,y_prob_rf))
```

```
Precision: 0.8383838383838383
Recall: 0.9764705882352941
F1 Score: 0.9021739130434783
ROC-AUC: 0.7955108359133128
```

```
results = pd.DataFrame({
    'Model':['Logistic Regression',
             'Random Forest'],
    'Precision':[
        precision_score(y_test,y_pred_lr),
        precision_score(y_test,y_pred_rf)
    ],
    'Recall':[
        recall_score(y_test,y_pred_lr),
        recall_score(y_test,y_pred_rf)
    ],
    'F1 Score':[
        f1_score(y_test,y_pred_lr),
        f1_score(y_test,y_pred_rf)
    ],
    'ROC-AUC':[
        roc_auc_score(y_test,y_prob_lr),
        roc_auc_score(y_test,y_prob_rf)
    ]
})
```

})

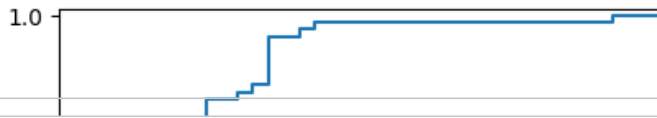
results

	Model	Precision	Recall	F1 Score	ROC-AUC	
0	Logistic Regression	0.863158	0.964706	0.911111	0.868731	
1	Random Forest	0.838384	0.976471	0.902174	0.795511	

Next steps:

[Generate code with results](#)[New interactive sheet](#)

```
RocCurveDisplay.from_predictions(  
    y_test,  
    y_prob_lr  
)  
  
RocCurveDisplay.from_predictions(  
    y_test,  
    y_prob_rf  
)  
  
plt.show()
```



```
feature_names = (
    numeric_features.tolist() +
    list(
        preprocessor.named_transformers_['cat']
        .named_steps['encoder']
        .get_feature_names_out(categorical_features)
    )
)

importance = pd.DataFrame({
    'Feature': feature_names,
    'Importance': rf.feature_importances_
})

importance = importance.sort_values(
    by='Importance',
    ascending=False
)

print(importance.head(10))
```

	Feature	Importance
4	Credit_History	0.127716
0	ApplicantIncome	0.082924
2	LoanAmount	0.074080
1	CoapplicantIncome	0.052006
500	Property_Area_Semiurban	0.031593
3	Loan_Amount_Term	0.022725
500	Property_Area_Rural	0.021012
500	Dependents_0	0.017729
500	Property_Area_Urban	0.017330